Recapitulare

Alexandru Olteanu

Universitatea Politehnica Bucuresti
Facultatea de Automatică si Calculatoare, Departamentul Calculatoare
alexandru.olteanu@cs.pub.ro

OOP, 2018

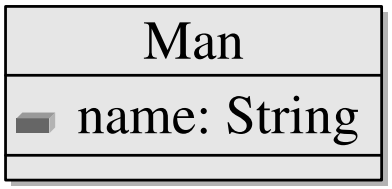




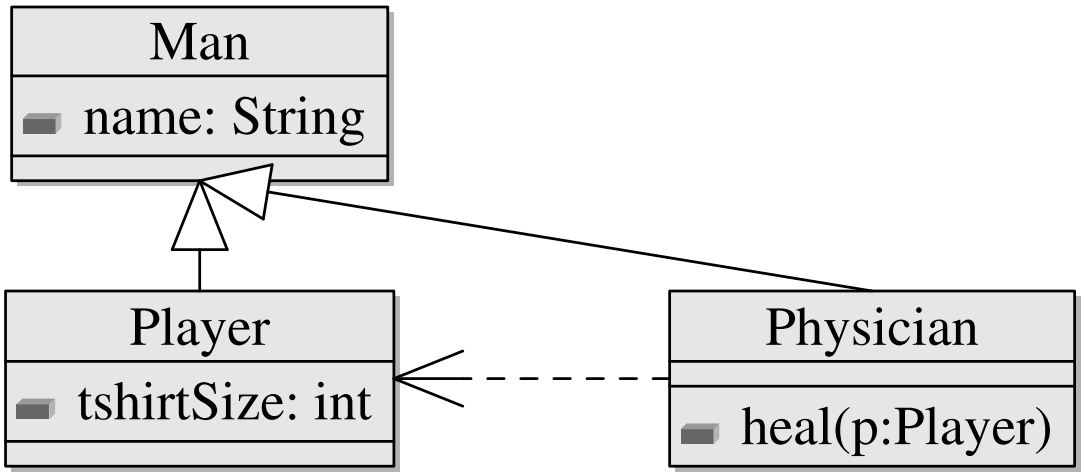


Universitatea
Politehnica
Bucureşti

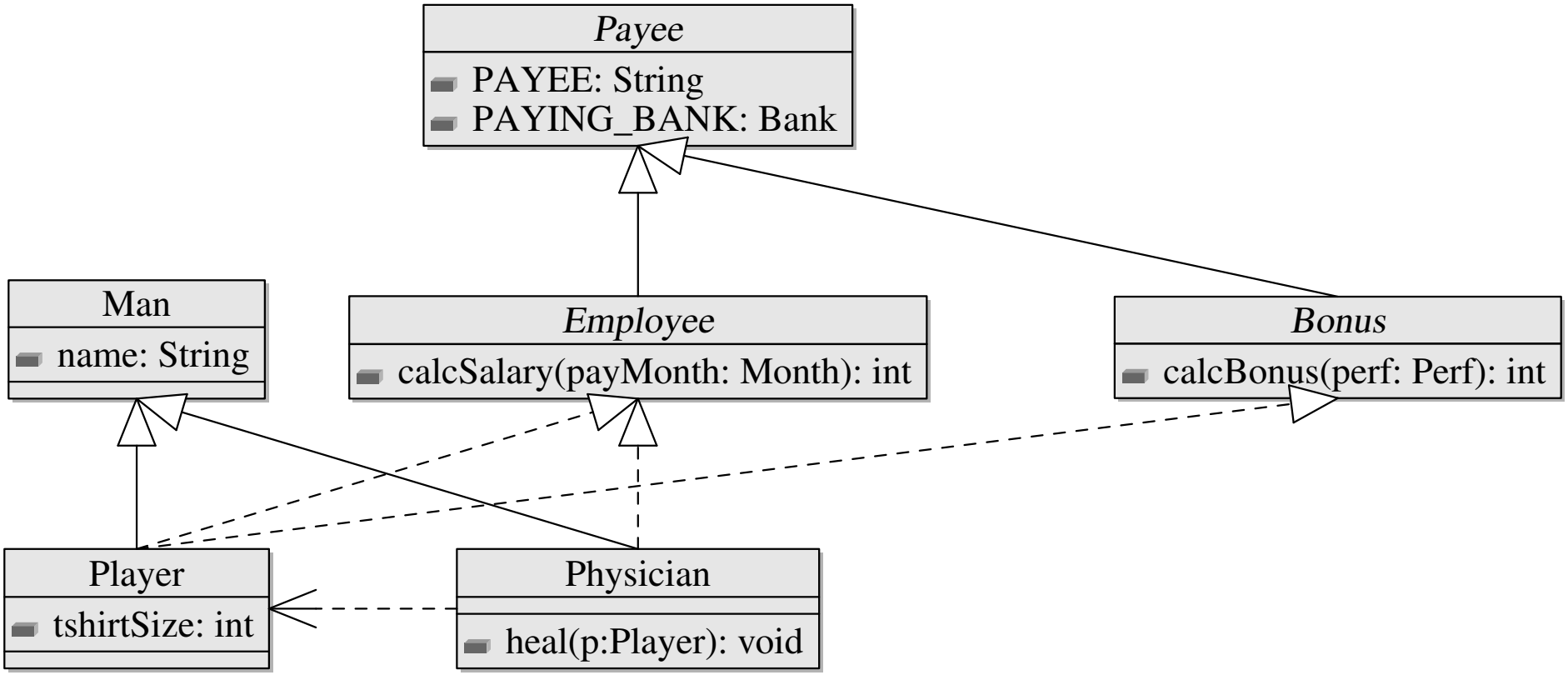
Curs 2: ce contine o clasa



Curs 3: relatia dintre clase



Curs 4: clase abstracte si interfete



Curs 5: polimorfism



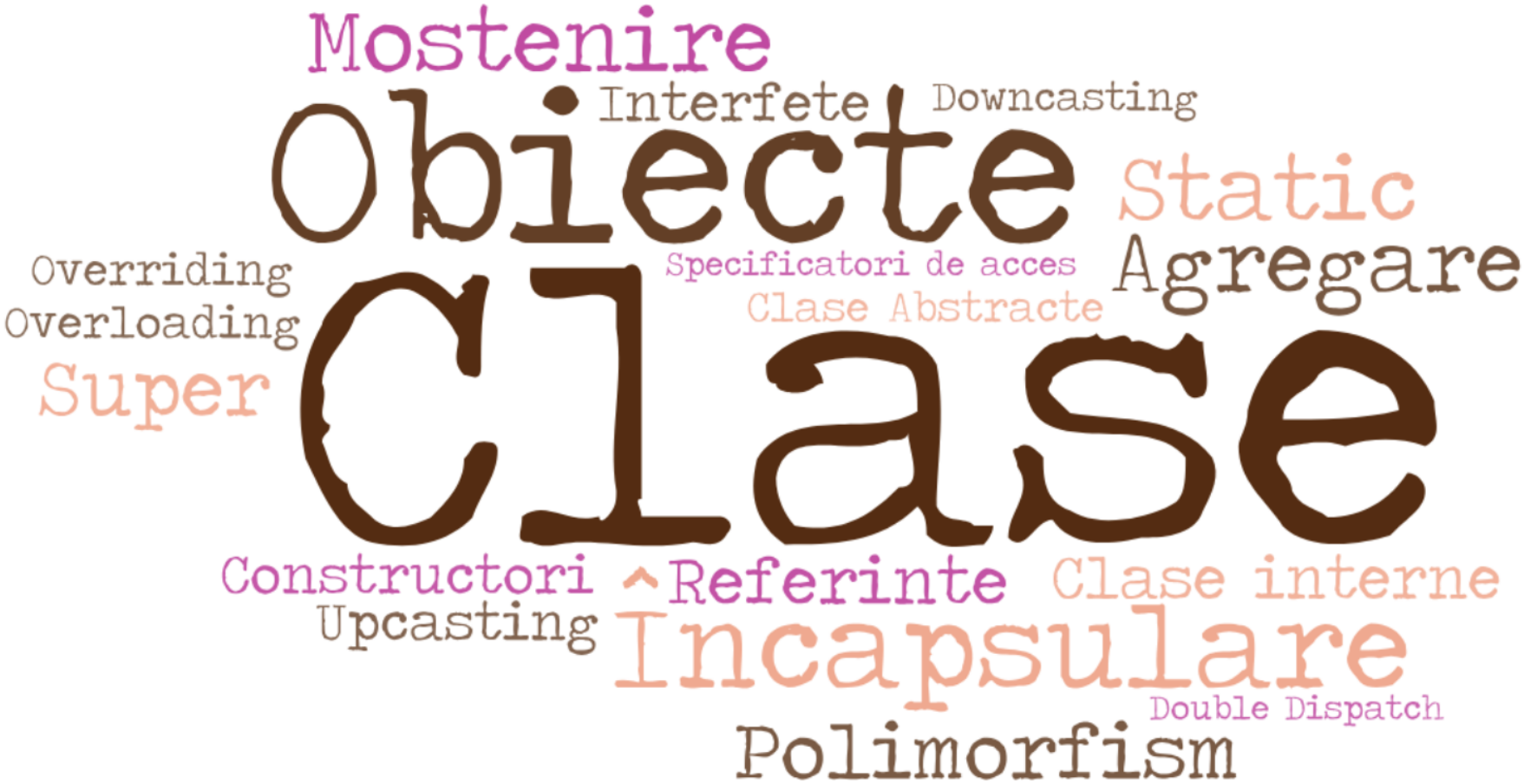
Curs 6: Object Oriented Design Principles

SOLID:

- SRP: Single Responsibility Principle
- OCP: Open-Closed Principle
- LSP: Liskov Substitution Principle
- ISP: Interface Segregation Principle
- DIP: Dependency Inversion Principle

KISS, YAGNI, DRY

Partea 1: Bazele teoretice ale OOP



```
public interface Collection<E> extends Iterable<E> {
  boolean add(E e);
  boolean addAll(Collection<? extends E> c);
  <T> T[] toArray(T[] a);
}
```

Curs 8: JNI, Exceptii, Testare

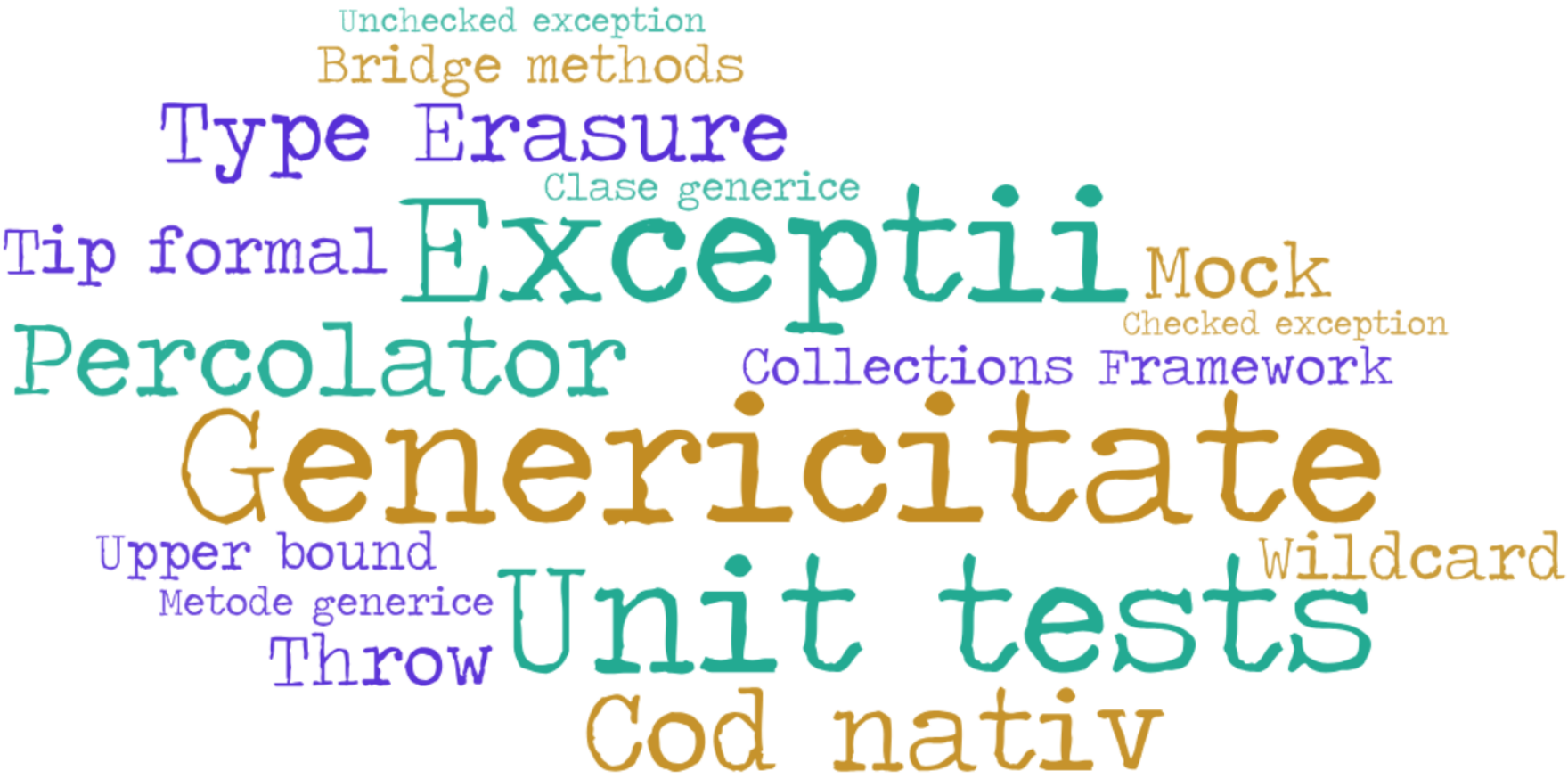
JNI: Native code vs Managed code

Exceptions



Testare: JUnit, Mock

Partea a 2-a: Antract pragmatic



Curs 9: Design Patterns 1

Clasificare Design Patterns:

- *creational patterns*: Singleton
- *structural patterns*: Adapter
- *behavioral patterns*: Visitor
- *architectural patterns*: Client-Server, Three-Tier

Frameworks - Inversion of Control implementation:

- Dependency Injection
- Strategy pattern

Curs 10: Design Patterns 2

Clasificare Design Patterns:

- *creational patterns*: Factory
- *structural patterns*
- *behavioral patterns*: Iterator, Observer
- *architectural patterns*: Publisher-Subscriber vs Request-Response

Frameworks:

- CRUD, REST, MVC
- Repository
- POJO, Java Beans, Entities, DTOs

Curs 11: Breaking the Patterns

Reflection:

- Introspection
- Reflection: how-to, avantaje, dezavantaje
- Reflection vs Encapsulation, vs Generics, vs Singleton

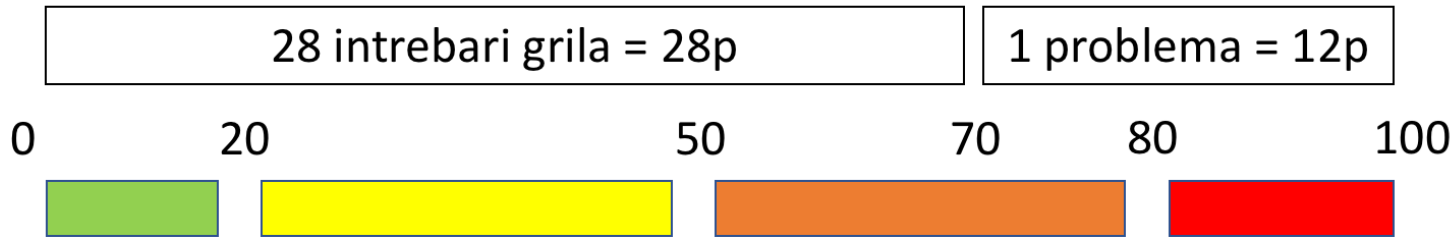
AntiPatterns:

- God Object, Spaghetti Code, Boilerplate Code
- Singleton, Factory

Partea a 3-a: Design Patterns



Structura examenului



D'oh



I can do this



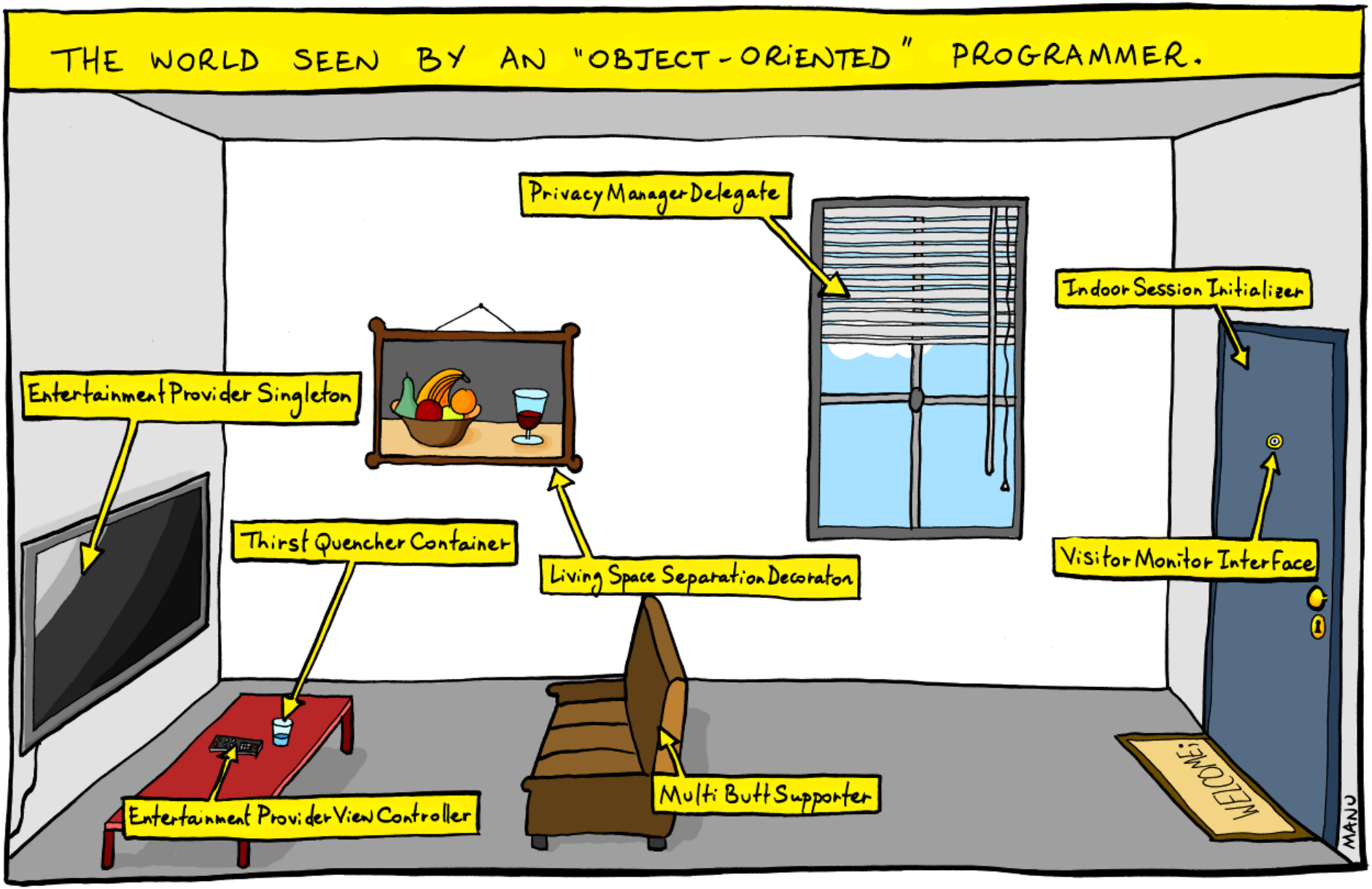
Bring it on



You are the one



What we expect from you ...



What you are probably thinking ...

